



Examen: Trivia del Conocimiento con JavaScript



Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación web interactiva de preguntas y respuestas (Trivia)**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación debe permitir:

1. Consultar las categorías y preguntas disponibles desde el backend.
2. Renderizar la pregunta actual y sus opciones de respuesta dinámicamente en el DOM.
3. Evaluar la respuesta elegida por el usuario, otorgar puntos y dar retroalimentación visual (acierto/error).
4. Persistir el puntaje e historial de respuestas en el navegador mediante `localStorage`.
5. Permitir la limpieza del historial y reinicio de la partida.



Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/categorias</code> y <code>/api/preguntas</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de trivia con fetch y async await</code>
#3	Renderizar dinámicamente las categorías en el selector y la pregunta con sus opciones en el DOM.	<code>feat(js): renderizar preguntas y opciones dinamicamente en el dom</code>
#4	Implementar eventos de click en las opciones, verificación de respuesta, feedback y actualización del puntaje.	<code>feat(js): implementar seleccion de respuesta y calculo de puntaje</code>
#5	Persistir las partidas en <code>localStorage</code> , renderizar el historial y permitir su limpieza con <code>#btn-limpiar-historial</code> .	<code>feat(js): persistir y gestionar historial de trivia en localStorage</code>



Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/categorias` : Devuelve la lista de categorías únicas.
- `GET http://localhost:3000/api/preguntas` : Devuelve las preguntas (soporta query param opcional `?categoria=Ciencia`).

- **POST `http://localhost:3000/api/validar`** : Valida `{ idPregunta, respuestaSeleccionada }` y devuelve `{ correcta, respuestaCorrecta, textoRespuestaCorrecta }`.

Para iniciar el servidor backend:

```
npm start
```

2. Elementos Clave del DOM

- **`#select-categoria`** : `<select>` para filtrar o cargar preguntas por categoría.
- **`#btn-iniciar`** : Botón para iniciar o cargar la lista de preguntas.
- **`#pregunta-card`** : Tarjeta donde se visualiza la pregunta actual (remover la clase `hidden`).
- **`#categoria-badge`** : Badge de categoría de la pregunta.
- **`#pregunta-texto`** : Encabezado donde se inserta el texto de la pregunta.
- **`#opciones-container`** : Contenedor donde se insertan los botones de cada opción.
- **`#feedback-mensaje`** : Contenedor de mensaje de acierto o error.
- **`#btn-siguiente`** : Botón para avanzar a la siguiente pregunta.
- **`#puntaje-actual`** : Marcador que refleja los puntos acumulados.
- **`#historial-lista`** : Lista `<ul>` donde se registran los aciertos y fallos de cada respuesta.
- **`#btn-limpiar-historial`** : Botón que limpia el historial y reinicia el puntaje.

3. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'trivia_historial'`
- **Estructura:** Arreglo de objetos con `{ pregunta, correcta, fecha }`.
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para leer.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.